Consuma take home assignment

Distributed Multi-Modal GenAI Pipeline

This assessment evaluates your ability to design and build resilient, distributed backend systems from first principles. Since AI coding assistants are prevalent, we will not evaluate you on boilerplate code. Instead, we are evaluating your **architectural choices, state management across boundaries, edge-case handling, and system reliability.**

Part 1: The Scenario & Architecture Constraints

You are building the core asynchronous engine for a Multi-Modal Generation Platform. Users upload large text manuscripts, and the system outputs a fully produced audio drama.

- **The Restriction:** You are **NOT** allowed to use managed workflow orchestrators (e.g., Temporal, Airflow, Step Functions, Celery). You must choreograph the pipeline using core infrastructure primitives.
- **The Tech Stack:** Python or Go.

Part 2: Execution MVP

Build a locally runnable, distributed microservices pipeline using docker-compose.

Required Infrastructure Components (via Docker Compose)

1. **API Gateway:** A lightweight web server (e.g., FastAPI, Gin) to accept jobs and serve status updates.
2. **Worker Node(s):** A separate service that consumes messages and processes the pipeline.
3. **Message Broker:** RabbitMQ or Kafka for event-driven choreography.
4. **State Database:** Postgres or MySQL for tracking job/task state.
5. **Cache/Lock Store:** Redis for rate-limiting, idempotency, and distributed locking.
6. **Object Storage:** MinIO (local S3 alternative) to store intermediate "files."

The Pipeline Steps (Simulated)

Instead of calling real AI vendors, simulate the APIs using sleep functions and randomized failure injections.

1. **Ingestion:** The API Gateway receives a manuscript string, saves it as a .txt file in MinIO, creates a database record (PENDING), and publishes a JobCreated event to the broker.
2. **Text Parsing (Simulated LLM):** Worker consumes the event, downloads the file, and "parses" it. *Simulate a 15% 500-Internal-Server-Error rate here to test your retry logic.*
3. **TTS Generation (Simulated Vendor):** * *Constraint A (Concurrency):* The vendor only allows **3 concurrent requests globally**. You must implement a distributed lock/semaphore (using Redis) across your workers.

    - *Constraint B (Cost/Idempotency):* If the exact same text block is sent twice, you must not hit the "vendor" again. Implement a caching layer that returns a previously generated MinIO URL if the hash matches.
4. **Stitch & Notify:** The worker "combines" the audio files, uploads the final asset to MinIO, updates the DB to COMPLETED, and fires a webhook (or logs a local event) to simulate notifying the user.

Critical Resilience Requirements

Requirement	Expected Implementation
Idempotent Consumers	If the Message Broker accidentally delivers the same JobCreated event twice, your worker must not process the job twice or corrupt the database.
Dead Letter Queue (DLQ)	If a specific manuscript consistently fails the TTS step (e.g., simulated "Poison Pill" payload), it must be routed to a DLQ after 3 retries with exponential backoff, without blocking the rest of the queue.
Crash Recovery	If a worker container is forcefully killed (docker kill) mid-processing, the message should not be lost. Another worker must pick it up after a timeout, or it must resume upon restart.